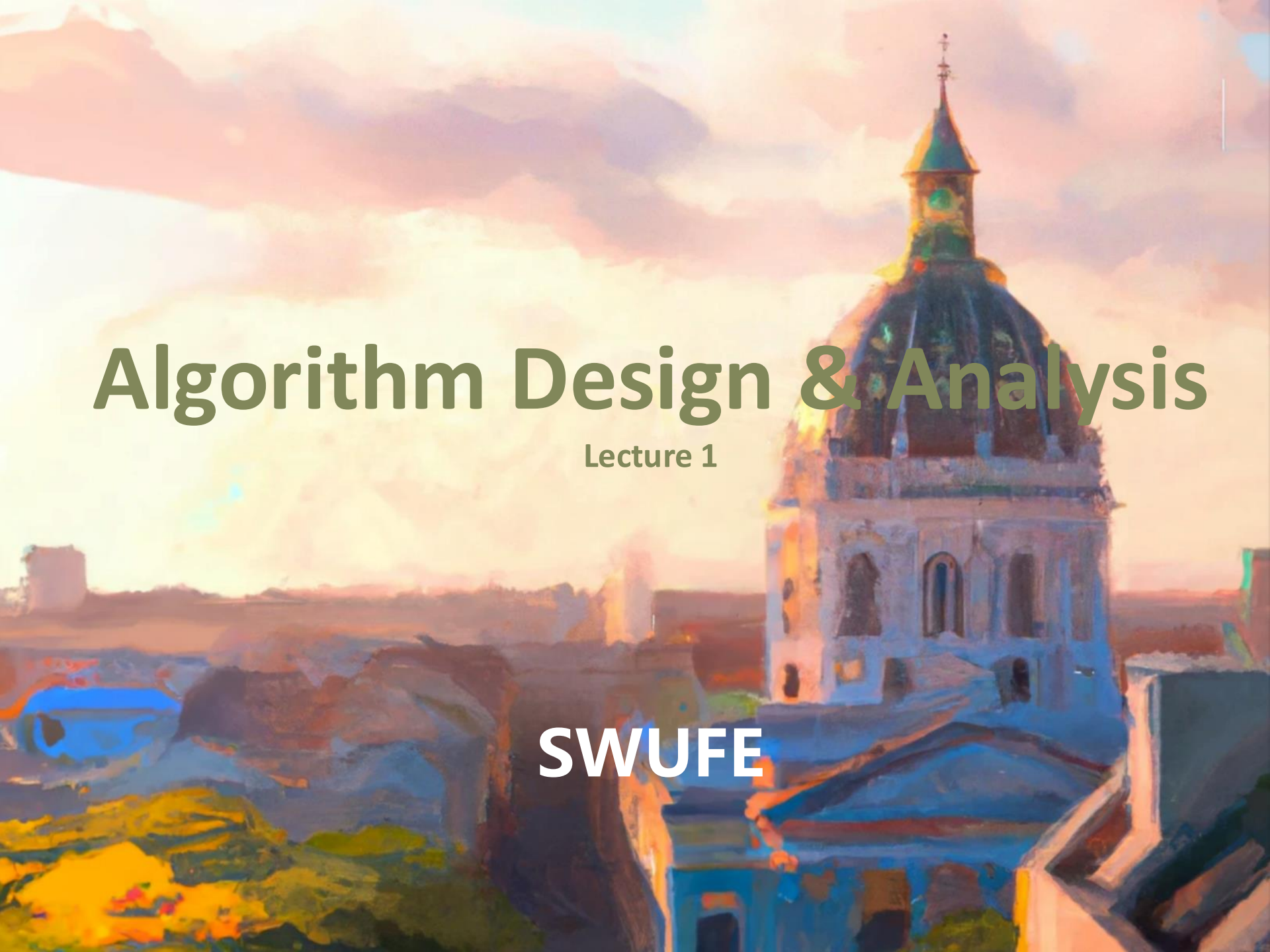


The background of the slide is a painting of a cityscape at sunset. A large, ornate domed church with a green roof and a spire is the central focus on the right side. The sky is filled with soft, colorful clouds in shades of orange, pink, and purple. The foreground shows the rooftops and walls of other buildings in the city, rendered in a painterly style with visible brushstrokes.

Algorithm Design & Analysis

Lecture 1

SWUFE



姓名：王海林

研究领域：

信息抽取、关系抽取、多模态语义结合

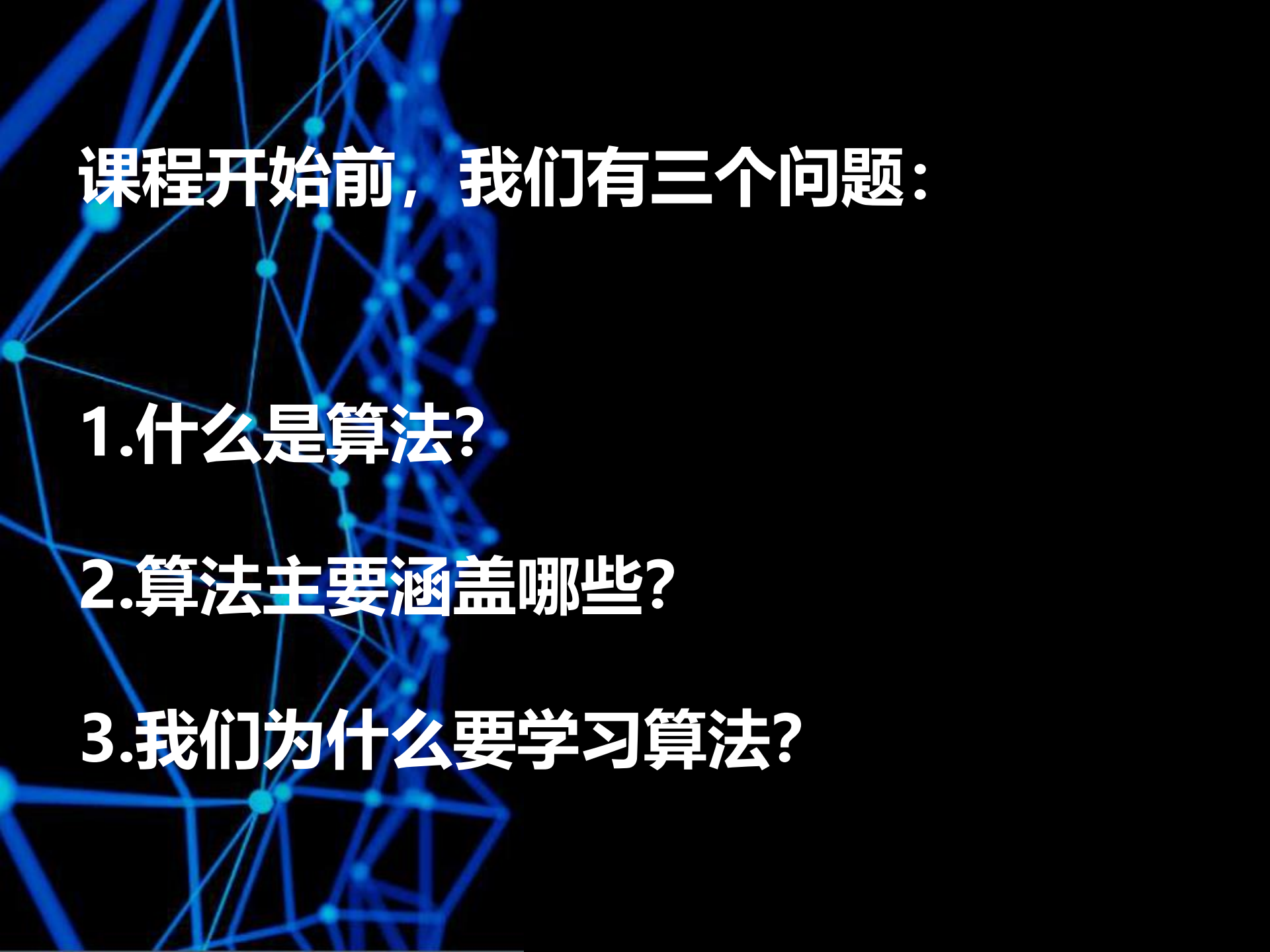
办公室：经世楼D205

电话：18981225300

邮箱：wanghl@swufe.edu.cn

QQ：2470548147

QQ Group：914924507

An abstract graphic in the background consisting of a complex network of blue lines and dots, resembling a molecular structure or a data network, set against a dark blue background.

课程开始前，我们有三问题：

1.什么是算法？

2.算法主要涵盖哪些？

3.我们为什么要学习算法？

Question1: 什么是算法?



nuScenes

ICRA2020 国际自动驾驶3D目标检测挑战赛冠军

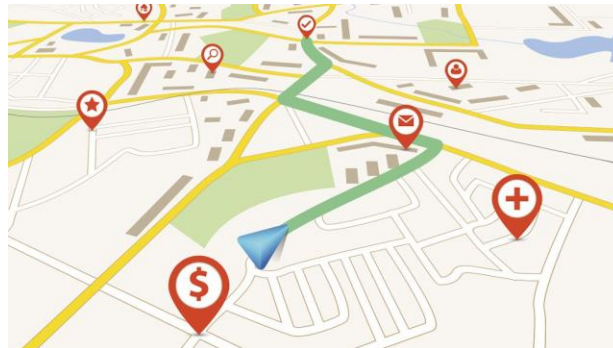
nuScenes is a public large-scale dataset for autonomous driving. It enables researchers to study challenging urban driving situations using the full sensor suite of a real self-driving car.

定义：解决问题的方法即算法

常见问题：排序记录、搜索数据项、计算素数
网约车调度、路线导航、投资产品组合

算法分类：计算机中算法通常由有限条指令构成

有限确定性算法 有限非确定性算法 无限算法



常规的：对话生成、图像生成、声音识别....

海面战舰识别、导弹袭击预警、社会舆情趋势分析、选举预测

—— 经世济民，孜孜以求 ——



计算机视觉是算法么？

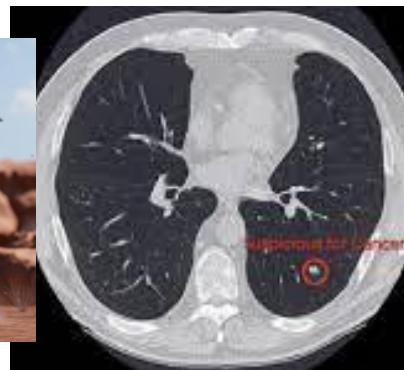
什么是算法?(3/X)

这些是算法么？

宇树科技机器人



癌症检测/生物医药研发



自动驾驶



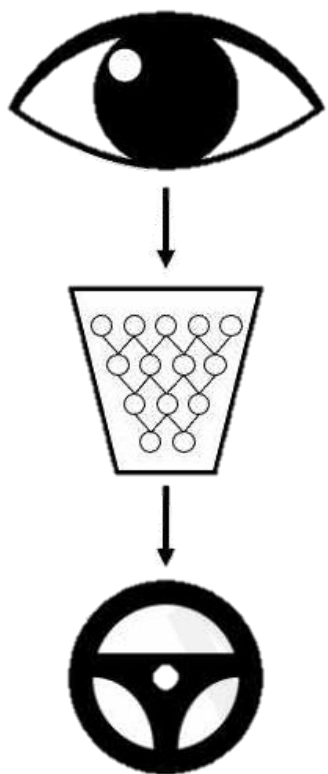
移动计算



—— 经世济民，孜孜以求 ——

什么是算法?(4/X)

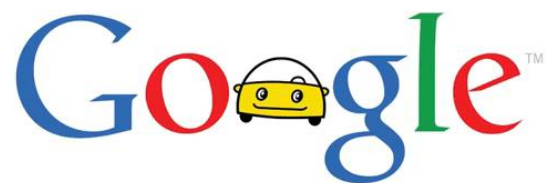
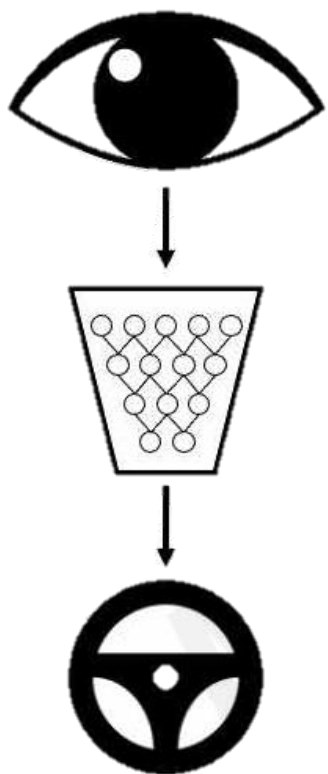
宇树科技视频



—— 经世济民、孜孜以求 ——

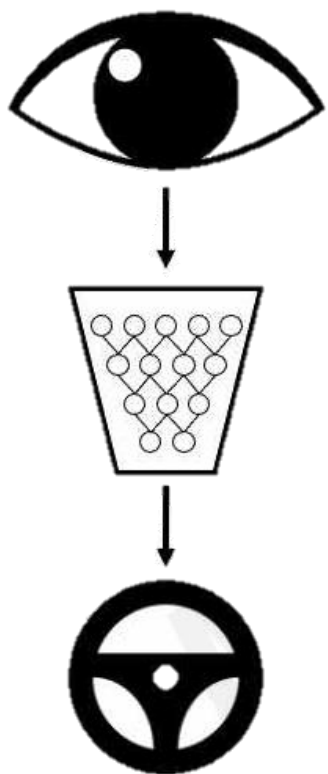


自动驾驶





飞行器捕捉



—— 经世济民，孜孜以求 ——

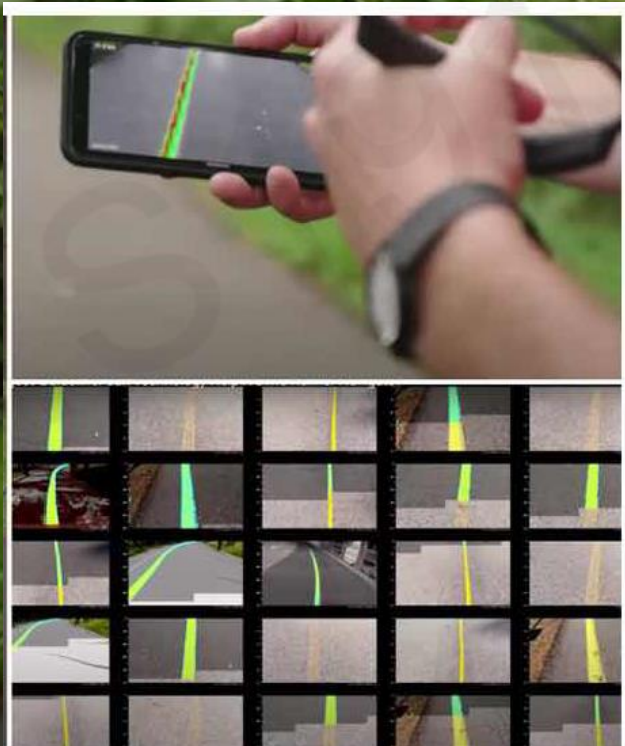


伪造音频





影响之便捷性



2020年谷歌开发的人工智能应用程序帮助“born to run”的盲人完成单人5公里跑
A smartphone camera picks up a painted "guideline" on a running track. An app detects the runner's position and gives audio guidance through an earpiece.



影响之便捷性



AmazonGo Video

—— 经世济民，孜孜以求 ——



爱卷毛的学习桑  优酷

任世所共、欣欣一好



大模型演示、Sora、 Diffusion

SORA实测|3分钟速览|高燃混剪|AI生成视频|含输入|完整版4k高清

OpenAI最新Sora视频合集05，太疯狂了！

Stable diffusion3

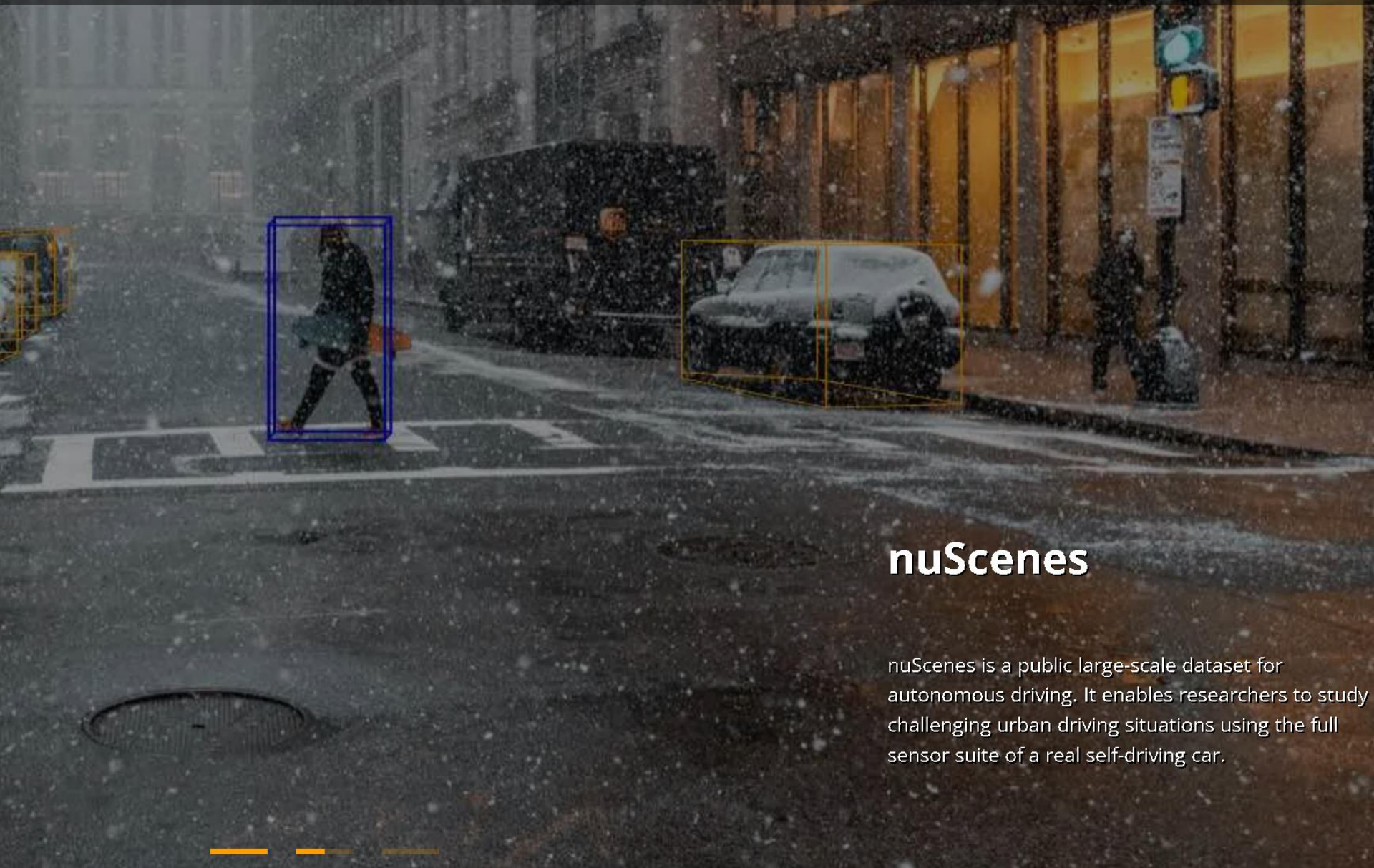
Deepseek 接入诊疗

最新的海螺AI试用

Deepseek 接入政务云

《千秋诗颂》第一集来啦！

Question1: 算法主要涵盖哪些?



nuScenes

nuScenes is a public large-scale dataset for autonomous driving. It enables researchers to study challenging urban driving situations using the full sensor suite of a real self-driving car.



1.常用算法

- **数值算法：**随机化、因子分解、素数问题、数值积分
- **数据结构：**数组、链表、树、网络
- **高级结构：**堆、平衡树、B树
- **排序和查找：**选择排序、插入排序、快速排序
- **网络算法：**最短路径、生成树、拓扑排序、流量计算

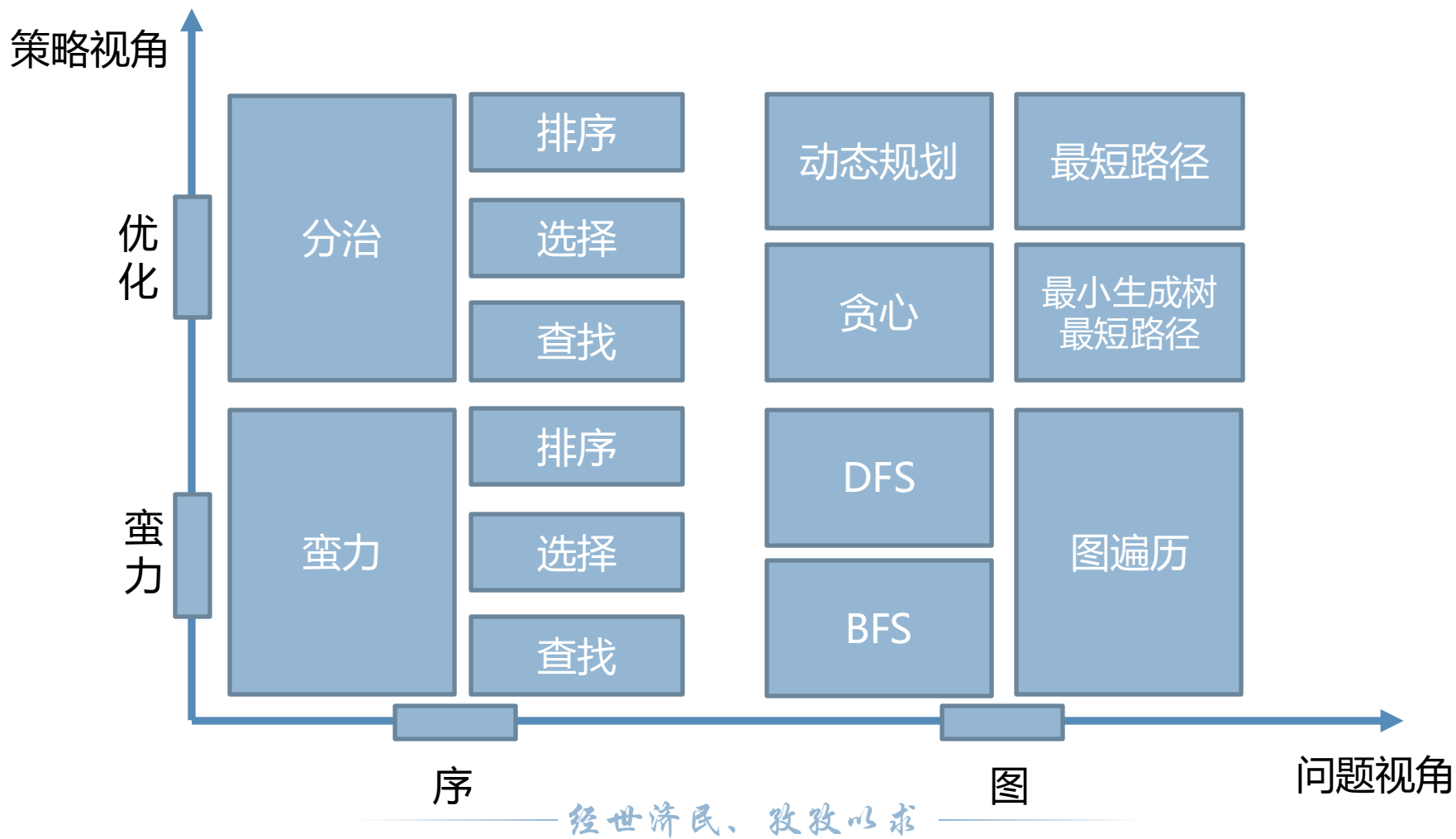


2.常用算法求解方法

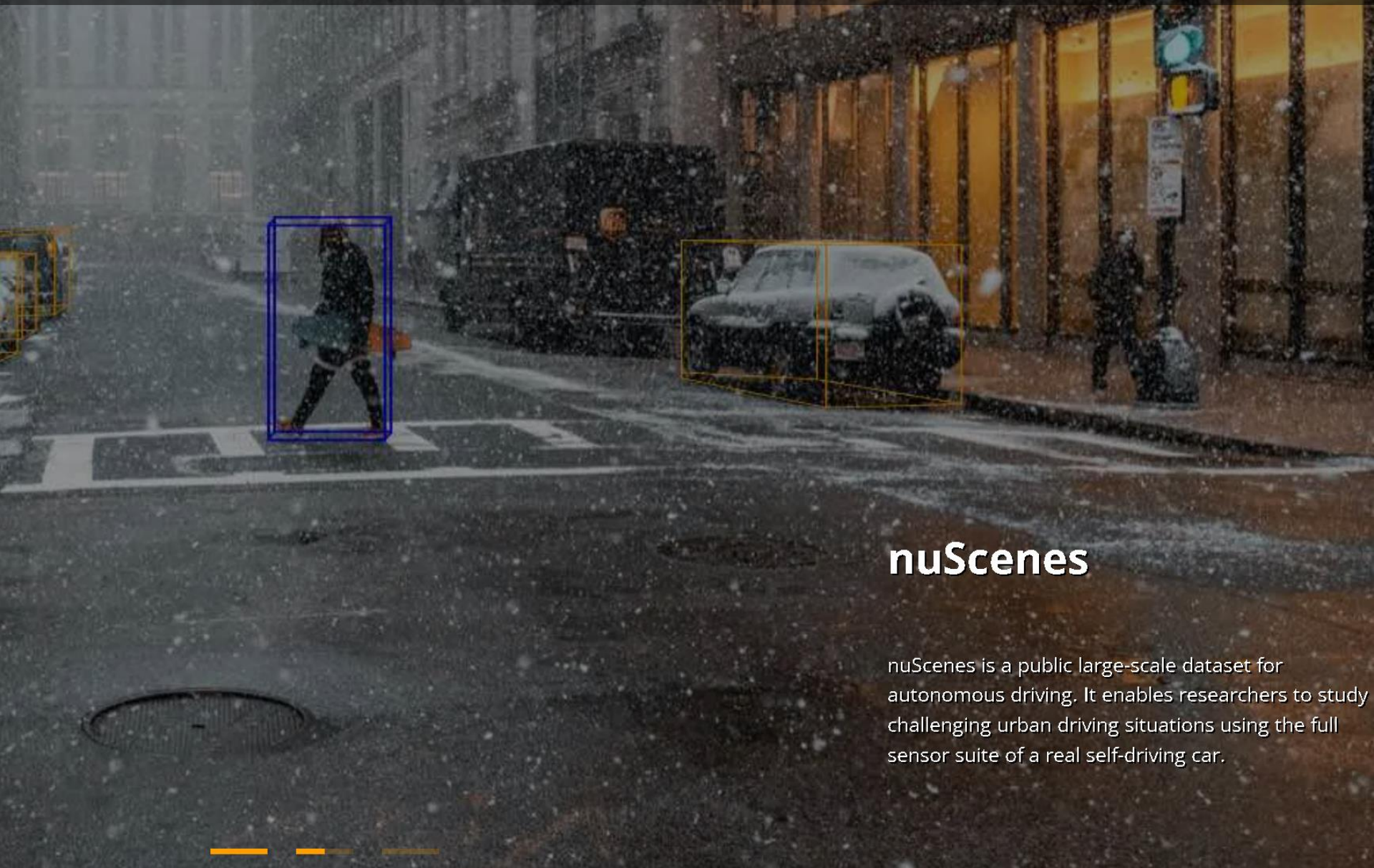
- 暴力求解、穷举搜索
- 分而治之
- 回溯法
- 递归法
- 分支限界法
- 贪婪算法和爬山算法
- 限制范围
- 启发式算法



3. 算法总体二维视角



Question3: 我们为什么要学习算法?



nuScenes

nuScenes is a public large-scale dataset for autonomous driving. It enables researchers to study challenging urban driving situations using the full sensor suite of a real self-driving car.



我们为什么要学习算法？

首先， 算法提供了有用的**工具**，我们可以用这些工具解决问题，比如排序、最短路径，理解这些算法，有助于我们在程序设计中决定采用哪种适合的数据结构；

其次， 算法可以**锻炼我们的大脑**，就像力量训练、技巧训练（足球、篮球），多加练习都有助于提高我们的能力；

最后， 算法**具有很强的趣味性**，可以使人获得满足感，当实现一个算法，并取得预期结果时，我仍然能够感觉到惊喜；

An abstract graphic on the left side of the slide, consisting of a complex network of blue lines and dots, resembling a molecular structure or a data network, set against a dark blue background.

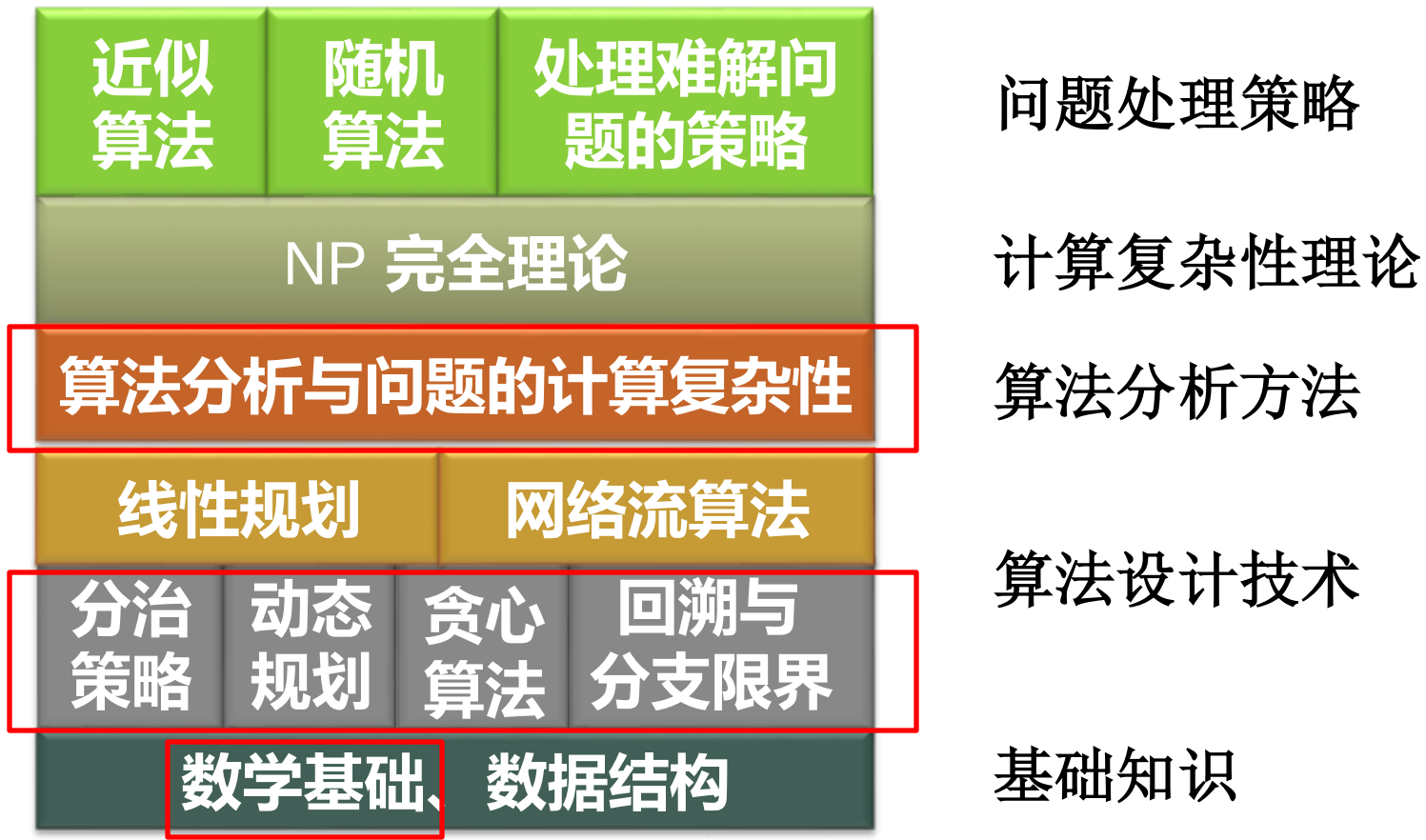
同学们最关心的问题：

1.本门课主要涉及哪些内容？

2.课程如何考核？



1.本门课主要涉及哪些内容?





2.课程如何考核?

- 翻转课堂（30%）：4部分内容，每部分内容需要3组讲解，每组3人，讲解一个对应章节算法问题，全班互评。
- 课后作业（20%）：4次作业每次5分
- 考勤（10%）：按照点名次数计算
- 大作业（10%）:(配合着PPT)
- 报告ppt（占20%）(15-16周展示)

An abstract graphic on the left side of the slide, consisting of a complex network of blue lines and dots. The dots are small circles, and the lines are thin, connecting the dots in a web-like structure. The overall shape is irregular and extends from the top left towards the bottom center.

知识回顾:

1.两个简单算法设计例子

2.常见算法介绍



1.两个简单算法设计例子

01

调度问题

02

投资问题



调度问题

贪心法的解

问题建模

贪心正确么

不一定正确

实例

算法设计

几个例子

例1.1 调度问题

问题：有 n 项任务，每项任务加工时间已知。

从0时刻开始陆续安排到一台机器上加工。

每个任务的完成时间是从0时刻到任务加工截止时间

求：总完成时间（所有任务完成时间之和）最短的安排方案

实例

任务集合 $S = \{1, 2, 3, 4, 5\}$

加工时间： $t_1 = 3, t_2 = 8, t_3 = 5, t_4 = 10, t_5 = 15$



调度问题

贪心法的解

问题建模

贪心正确么

不一定正确

实例

算法设计

贪心法的解

算法：按加工时间(3, 8, 5, 10, 15)从小到大安排

解：1, 3, 2, 4, 5



总周转时间：

$$\begin{aligned} t &= 3 + (3 + 5) + (3 + 5 + 8) + (3 + 5 + 8 + 10) + (3 + 5 + 8 + 10 + 15) \\ &= 3 \times 5 + 5 \times 4 + 8 \times 3 + 10 \times 2 + 15 \\ &= 94 \end{aligned}$$

*求解方法

贪心策略：加工时间短的先做

如何描述这个方法？这个方法是否对所有的实例都能得到最优解？如何证明？这个方法的效率如何？



调度问题

贪心法的解

问题建模

贪心正确么

不一定正确

实例

算法设计

问题建模

几个例子

例1.1 调度问题

输入：任务集 $S=\{1, 2, \dots, n\}$,

第 j 项任务加工时间: $t_j \in \mathbb{Z}^+, j=1, 2, \dots, n$

输出：调度 I . $\longrightarrow$ S 的排列, $i_1, i_2, i_3, \dots, i_n$,

目标函数： I 的完成时间, $t(I) = \min\left\{\sum_{k=1}^n (n - k + 1)t_{i_k}\right\}$

解： I^* , 使得 $t(I^*)$ 达到最小, 即

$$t(I^*) = \min\{t(I) | I \text{ 为 } S \text{ 的排列}\}$$



调度问题

贪心法的解

问题建模

贪心正确么

不一定正确

实例

算法设计

贪心算法正确么

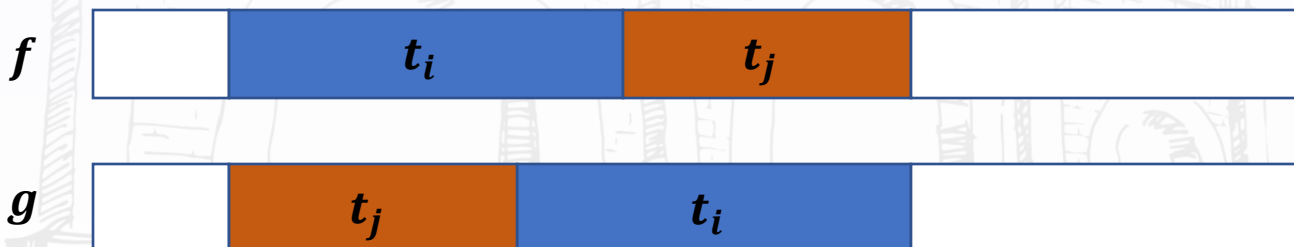
设计策略： 加工时间短的先做

算法： 根据加工时间从小到大排序，依次加工

算法正确性： 对所有输入实例都得到最优解？

证明： 假如调度 f 第 i, j 项任务相邻且有逆序，

即 $t_i > t_j$. 交换任务 i 和 j 得到调度 g ,



总完成时间 $t(g) - t(f) = t_j - t_i < 0$



调度问题

贪心法的解

问题建模

贪心正确么

不一定正确

实例

算法设计

贪心并不总是正确-实例

反例

有4件物品要装入背包，物品重量和价值如下：

标号	1	2	3	4
重量 w_i	3	4	5	2
价值 v_i	7	9	9	2

背包重量限制是6，问如何选择物品，使得不超重的情况下装入背包的物品价值达到最大？



调度问题

贪心法的解

问题建模

贪心正确么

不一定正确

实例

算法设计

贪心并不总是正确-实例

贪心法：单位重量价值大的优先，总重量不超过6，按照 $\frac{v_i}{w_i}$ 从大到小排序: 1, 2, 3, 4

$$\frac{7}{3} > \frac{9}{4} > \frac{9}{5} > \frac{9}{2}$$



调度问题

贪心法的解

问题建模

贪心正确么

不一定正确

实例

算法设计

贪心并不总是正确-实例

贪心法：单位重量价值大的优先，总重量不超过6，按照 $\frac{v_i}{w_i}$ 从大到小排序: 1, 2, 3, 4

$$\frac{7}{3} > \frac{9}{4} > \frac{9}{5} > \frac{9}{2}$$

贪心法的解：{1, 4}，重量5，价值9



调度问题

贪心法的解

问题建模

贪心正确么

不一定正确

实例

算法设计

贪心并不总是正确-实例

贪心法：单位重量价值大的优先，总重量不超过6，按照 $\frac{v_i}{w_i}$ 从大到小排序: 1, 2, 3, 4

$$\frac{7}{3} > \frac{9}{4} > \frac{9}{5} > \frac{9}{2}$$

贪心法的解：{1, 4}，重量5，价值9

更好的解：{2, 4}，重量6，价值11



调度问题

贪心法的解

问题建模

贪心正确么

不一定正确

实例

算法设计

算法设计

我们做了什么事情？

1. 问题建模

2. 选择什么算法？如何描述这个算法？

3. 这个方法是否对所有实例都得到最优解？

如何证明？

4. 如果不是，能否找到反例？

1. 问题建模

2. 设计算法

3. 验证算法



投资问题

问题建模

蛮力算法

实例计算

效率分析

例1.2 投资问题

问题：m元钱，投资n个项目.效益函数 $f_i(x)$ ，
表示第*i*个项目投*x*元的效益， $i = 1, 2, 3 \dots n$.

求如何分配每个项目的钱数使得效益最大？

实例：5万元，投资给4个项目，效益函数：

X	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24



投资问题

问题建模

蛮力算法

实例计算

效率分析

问题建模

输入： $n, m, f_i(x), i = 1, 2, \dots, n, x = 1, 2, \dots, m$

解： n 维向量 $\langle x_1, x_2, \dots, x_n \rangle$, x_i 是第 i 个项目的钱数，
使得下述条件满足：

$$\begin{aligned} \max \sum_{i=1}^n f_i(x_i), \\ \sum_{i=1}^n x_i = m, \quad x_i \in N \end{aligned}$$



投资问题

问题建模

蛮力算法

实例计算

效率分析

问题建模

输入： $n, m, f_i(x), i = 1, 2, \dots, n, x = 1, 2, \dots, m$

解： n 维向量 $\langle x_1, x_2, \dots, x_n \rangle$, x_i 是第 i 个项目的钱数，
使得下述条件满足：

目标函数

$$\max \sum_{i=1}^n f_i(x_i),$$

约束条件

$$\sum_{i=1}^n x_i = m, \quad x_i \in N$$



投资问题

问题建模

蛮力算法

实例计算

效率分析

蛮力算法

对所有满足下述条件的向量 $\langle x_1, x_2, \dots, x_n \rangle$

$$x_1 + x_2 + \dots + x_n = m$$

x_i 为非负整数, $i = 1, 2, \dots, n$

计算相应的效益

$$f_1(x_1) + f_2(x_2) + \dots + f_n(x_n)$$

从中确认效益最大的向量



投资问题

问题建模

蛮力算法

实例计算

效率分析

实例计算

X	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24



投资问题

问题建模

蛮力算法

实例计算

效率分析

实例计算

X	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

$$x_1 + x_2 + x_3 + x_4 = 5$$

$$s_1 = \langle 0, 0, 0, 5 \rangle, v(s_1) = 24$$



投资问题

问题建模

蛮力算法

实例计算

效率分析

实例计算

X	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

$$x_1 + x_2 + x_3 + x_4 = 5$$

$$s_1 = \langle 0, 0, 0, 5 \rangle, v(s_1) = 24$$

$$s_2 = \langle 0, 0, 1, 4 \rangle, v(s_2) = 25$$



投资问题

问题建模

蛮力算法

实例计算

效率分析

实例计算

X	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

$$x_1 + x_2 + x_3 + x_4 = 5$$

$$s_1 = \langle 0, 0, 0, 5 \rangle, v(s_1) = 24$$

$$s_2 = \langle 0, 0, 1, 4 \rangle, v(s_2) = 25$$

$$s_3 = \langle 0, 0, 2, 3 \rangle, v(s_3) = 32$$

...

$$s_{56} = \langle 5, 0, 0, 0 \rangle, v(s_{56}) = 15$$



投资问题

问题建模

蛮力算法

实例计算

效率分析

蛮力算法的效率分析

方程 $x_1 + x_2 + \cdots + x_n = m$ 的非负整数解

$\langle x_1, x_2, \dots, x_n \rangle$ 的个数估计：



投资问题

问题建模

蛮力算法

实例计算

效率分析

蛮力算法的效率分析

方程 $x_1 + x_2 + \cdots + x_n = m$ 的非负整数解

$\langle x_1, x_2, \dots, x_n \rangle$ 的个数估计:

可行的解表示成0-1序列 m 个1, $n-1$ 个0

1 ... 1 0 1 ... 1 0 ... 0 1 ... 1

x_1 个

x_2 个

x_n 个



投资问题

问题建模

蛮力算法

实例计算

效率分析

蛮力算法的效率分析

方程 $x_1 + x_2 + \cdots + x_n = m$ 的非负整数解

$\langle x_1, x_2, \dots, x_n \rangle$ 的个数估计:

可行的解表示成0-1序列 m 个1, $n-1$ 个0

1 ... 1 0 1 ... 1 0 ... 0 1 ... 1

x_1 个

x_2 个

x_n 个

$n = 4, m = 7$ 7万金额分成4份

示例

可行解 $\langle 1, 2, 3, 1 \rangle$

序列 **1011011101**



投资问题

问题建模

蛮力算法

实例计算

效率分析

蛮力算法的效率分析

序列个数是输入规模的指数函数

$$C(m + n - 1, m)$$

$$= \frac{(m + n - 1)!}{m! (n - 1)!}$$

$$= \Omega((1 + \epsilon)^{m+n-1})$$

斯特林公式



有没有更好的算法？



例子小结

问题求解的关键

- 建模：对输入参数和解给出形式化或半形式化的描述
- 设计算法：
 - 采用什么算法设计技术
 - 正确性——是否对所有的实例都得到正确的解
- 分析算法——效率



2.常见算法介绍

01

排序问题

02

计算复杂度

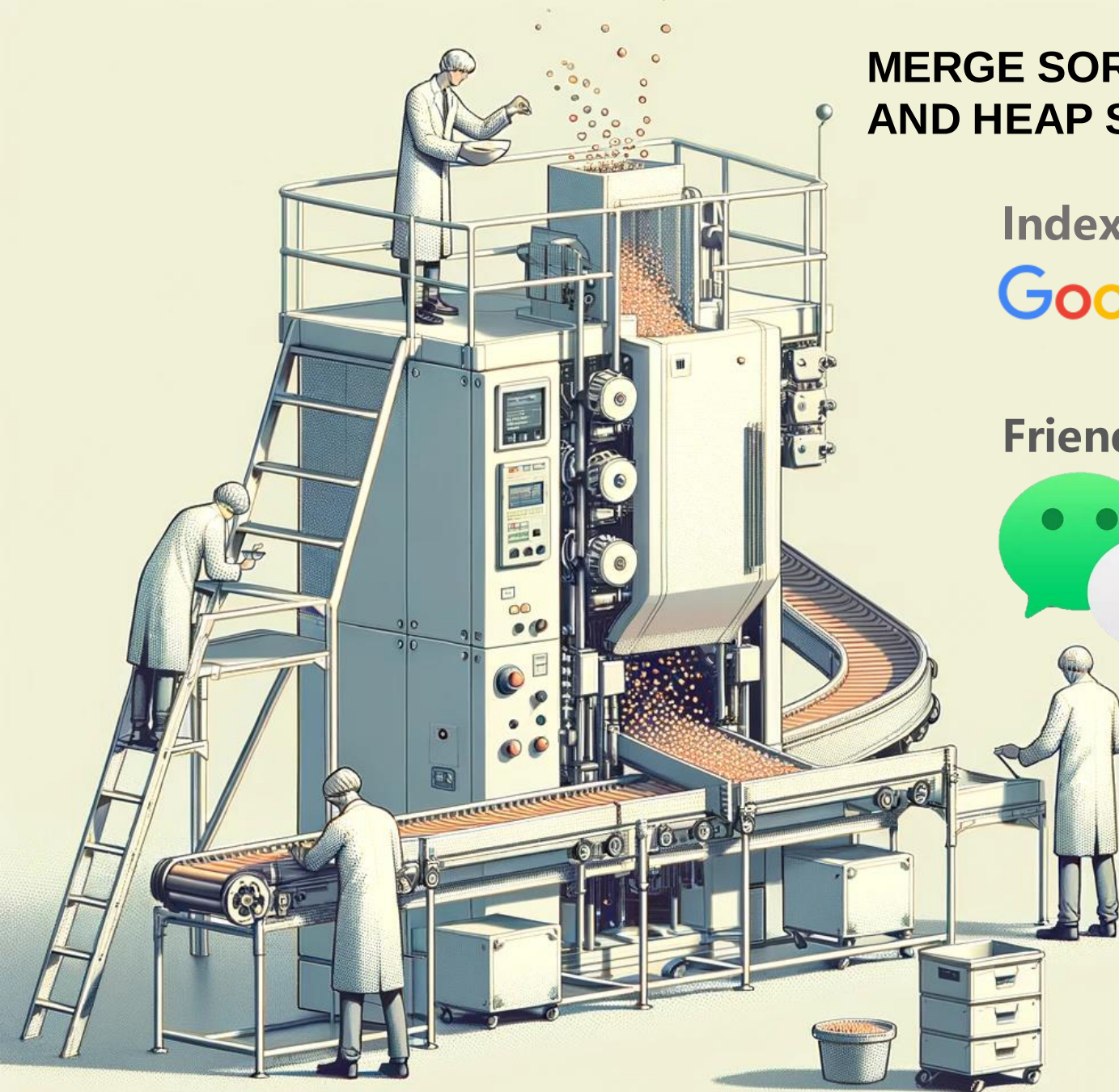
MERGE SORT, QUICK SORT AND HEAP SORT

Index Sorting

Google

Baidu 百度

Friends, Content sorting





排序效率

插入排序

插入实例

冒泡排序

冒泡实例

快速排序

二分归并

复杂度分析

排序问题算法的效率

以元素比较作为基本运算

算法	最坏情况下	平均情况下
插入排序	$O(n^2)$	$O(n^2)$
冒泡排序	$O(n^2)$	$O(n^2)$
快速排序	$O(n^2)$	$O(n \log n)$
堆排序	$O(n \log n)$	$O(n \log n)$
二分归并排序	$O(n \log n)$	$O(n \log n)$



插入排序

前面已经排好，插入2

输入

5	7	1	3	6	2	4
---	---	---	---	---	---	---

插入2

1	3	5	6	7	2	4
---	---	---	---	---	---	---

插入后

1	2	3	5	6	7	4
---	---	---	---	---	---	---

- 排序效率
- 插入排序
- 插入实例
- 冒泡排序
- 冒泡实例
- 快速排序
- 二分归并
- 复杂度分析



排序效率

插入排序

插入实例

冒泡排序

冒泡实例

快速排序

二分归并

复杂度分析

插入排序实例

输入	5	7	1	3	6	2	4
初始	5	7	1	3	6	2	4
插入7	5	7	1	3	6	2	4
插入1	1	5	7	3	6	2	4
插入3	1	3	5	7	6	2	4
插入6	1	3	5	6	7	2	4
插入2	1	2	3	5	6	7	4
插入4	1	2	3	4	5	6	7



排序效率

插入排序

插入实例

冒泡排序

冒泡实例

快速排序

二分归并

复杂度分析

冒泡排序一次巡回

巡回前

7	1	3	6	2	4
---	---	---	---	---	---

巡回

7	1	3	6	2	4
---	---	---	---	---	---

巡回后

1	3	6	2	4	7
---	---	---	---	---	---



冒泡实例

巡回前	5	8	1	3	6	2	4	7
巡回1	5	1	3	6	2	4	7	8
巡回2	1	3	5	2	4	6	7	8
巡回3	1	3	2	4	5	6	7	8
巡回4	1	2	3	4	5	6	7	8
巡回5	1	2	3	4	5	6	7	8

- 排序效率
- 插入排序
- 插入实例
- 冒泡排序
- 冒泡实例
- 快速排序
- 二分归并
- 复杂度分析



快速排序

排序效率

插入排序

插入实例

冒泡排序

冒泡实例

快速排序

二分归并

复杂度分析

交换1

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

5	8	1	3	6	2	4	8
---	---	---	---	---	---	---	---

交换2

5	4	1	3	6	2	8	7
---	---	---	---	---	---	---	---

划分

5	4	1	3	2	6	8	7
---	---	---	---	---	---	---	---

子问题

2	4	1	3	5	6	8	7
---	---	---	---	---	---	---	---



排序效率

插入排序

插入实例

冒泡排序

冒泡实例

快速排序

二分归并

复杂度分析

二分归并排序运行实例

划分

递归
排序

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

5	8	1	3	6	2	4	7
---	---	---	---	---	---	---	---

1	3	5	8	2	4	6	7
---	---	---	---	---	---	---	---

1	3	5	8	2	4	6	7
---	---	---	---	---	---	---	---



排序效率

插入排序

插入实例

冒泡排序

冒泡实例

快速排序

二分归并

复杂度分析

问题的计算复杂度分析

问题:

哪个排序算法效率最高?

是否可以找到更好的排序算法?

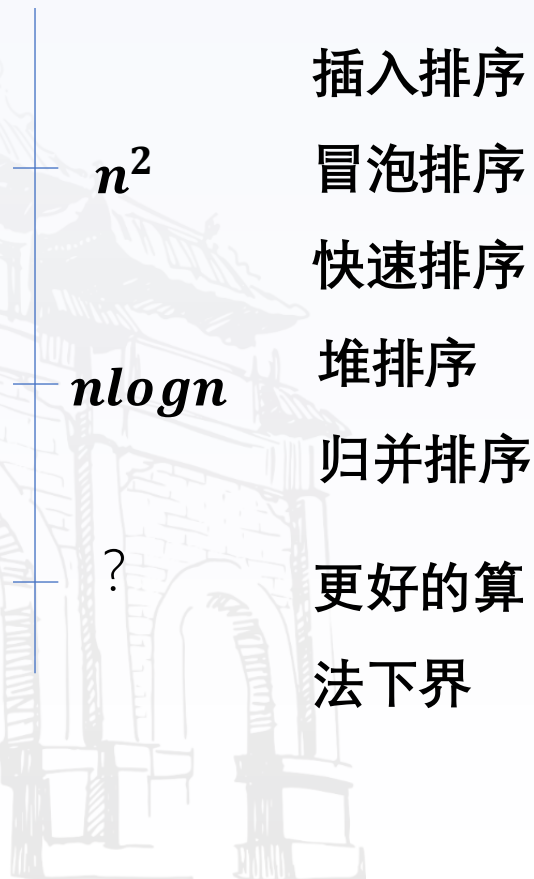
排序问题计算难度如何?

其他问题的计算复杂度

问题计算复杂度估计方法



哪个排序算法效率最高
排序问题的难度如何?
还有更快的算法么?



插入排序

冒泡排序

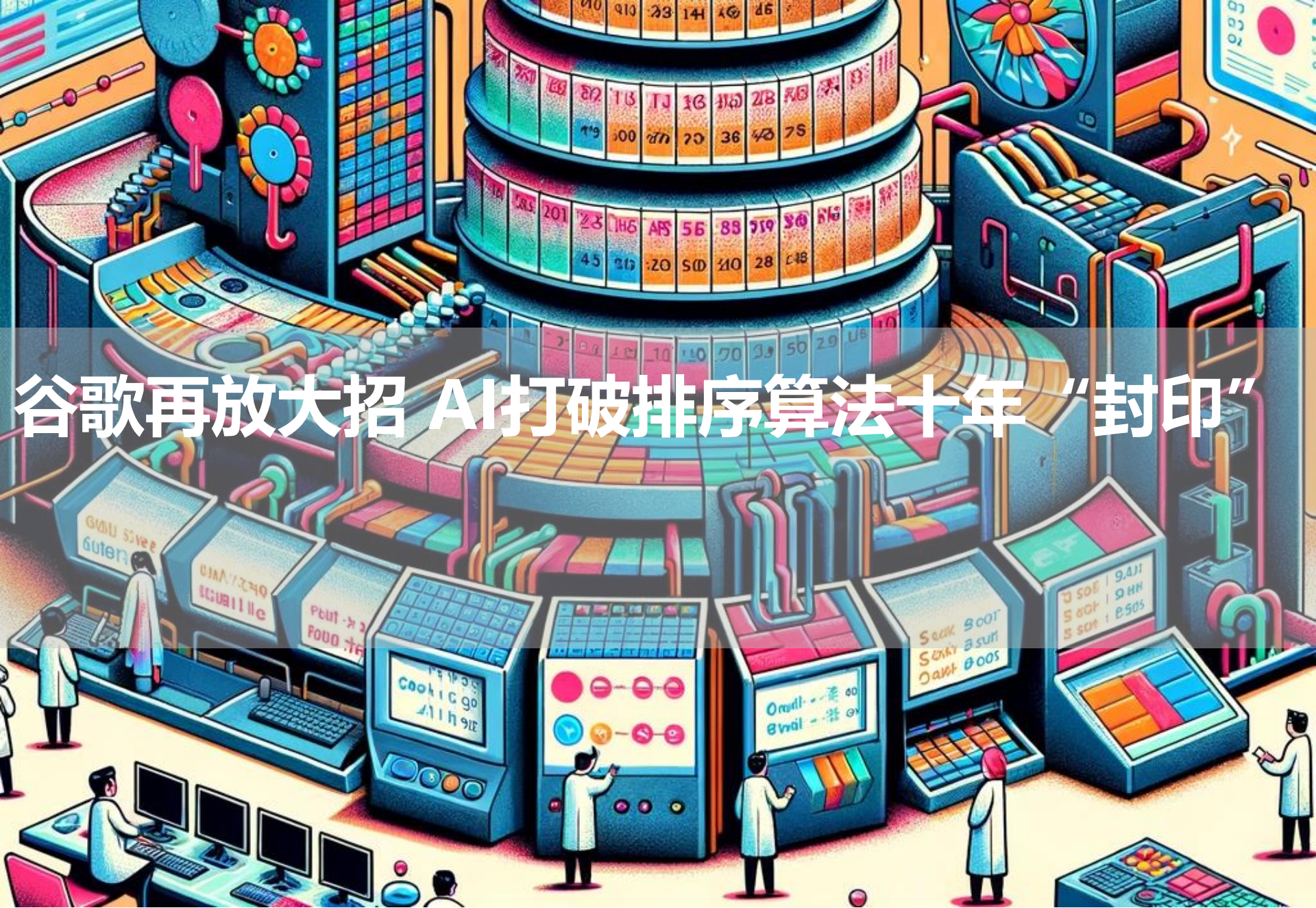
快速排序

堆排序

归并排序

更好的算

法下界



谷歌再放大招 AI打破排序算法十年“封印”



小结

几种排序算法简介

插入排序

冒泡排序

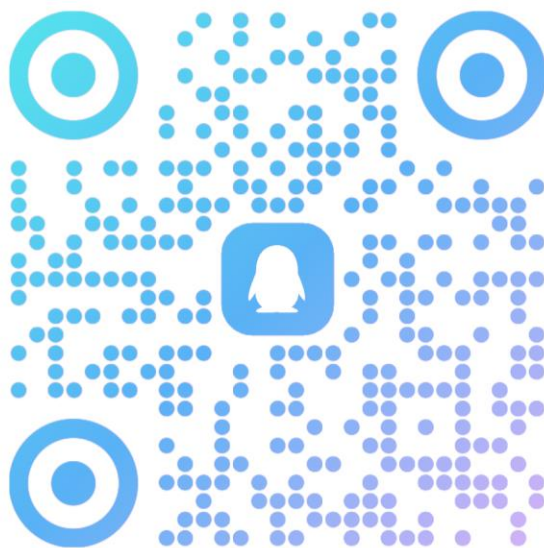
快速排序

归并排序



2024-2025下-算法...

群号: 980527384



扫一扫二维码，入群聊